

Chapter 79

GenAI, LLM & MLOps

Question Bank

Answer the LLM fundamentals, RAG design, and evaluation questions that come up in GenAI-adjacent Data Scientist interviews · reason correctly about embeddings, chunking, and retrieval, not just name the components · explain how a model actually survives production (registries, serving, drift...

Approved chapter · v1

Contents

Chapter 79. GenAI, LLM & MLOps Question Bank

79.1 LLM fundamentals

79.2 Basic-but-tricky GenAI questions

79.3 Retrieval-Augmented Generation (RAG)

79.4 Agents and tool use

79.5 Evaluating AI applications

79.6 MLOps: making models survive production

79.7 LLMOps

79.8 Full design cases

Common mistakes and how to spot them

In the real world: the RAG assistant that confidently cited a document that didn't say what it claimed

Tools

The project

Final-week revision list

Key terms

Where this leads

Chapter 79. GenAI, LLM & MLOps Question Bank

Part VIII — The Interview Playbook

You will learn to: answer the LLM fundamentals, RAG design, and evaluation questions that come up in GenAI-adjacent Data Scientist interviews · reason correctly about embeddings, chunking, and retrieval, not just name the components · explain how a model actually survives production (registries, serving, drift detection) · design a RAG assistant and a model-serving platform end to end, stating trade-offs out loud.

How this chapter is built. Same format as Chapters 70–78: every core question leads with a "Remember it as..." hook, a one-line answer, a compact tier table. Rapid-fire sections are scan tables. **Every numeric claim that could be computed without a live LLM API was actually run:** cosine similarity for retrieval, real text chunking, and a genuine statistical drift-detection test (KS test), all verified. Prompting technique, agent architecture, and vendor-specific behavior are conceptual by nature and are reasoned through carefully rather than asserted, since they can't be "run" the way a retrieval score can.

Learn it in pointers reference Chapters 54 (Generative AI & LLMs), 55 (RAG, Agents & Evaluation), 56 (MLOps), and 57 (LLMOps) of this book, at chapter level, since this chat doesn't have their approved text to check exact section numbers against.

A note on currency. GenAI tooling and model capabilities change fast. This chapter teaches durable concepts (how RAG works, why chunking matters, what drift detection measures) rather than specific model names, prices, or context-window sizes, which would go stale quickly; verify any current specifics against up-to-date sources before an actual interview.

79.1 LLM fundamentals

Q79-001 · What's a token, and why does a model's context window get measured in tokens, not words or characters?

Remember it as: A token is roughly a chunk of a word, not a whole word, which is exactly why "context window" numbers look bigger than you'd expect from word count alone.

Answer in one line: A **token** is the basic unit a language model actually processes, often a word, part of a word, or punctuation mark, produced by a tokenizer that breaks text into these sub-word pieces; models measure context (and often pricing) in tokens because that's the model's native unit of computation, and it doesn't map cleanly to word count, since a rare or long word can split into several tokens while a common short word is one.

Tier	What to say
Passes	"A token is basically a word" (a rough approximation, misses the sub-word splitting that actually matters for the answer)
Strong	Explains that a token is often a sub-word unit, and that this is precisely why token counts and word counts diverge, especially for uncommon words, code, or non-English text
Extra points	+ [Business] this divergence matters practically: a rough "words to tokens" ratio is commonly cited (English prose is often estimated around 0.75 words per token), but it's an approximation that breaks down for code, technical jargon, or other languages, and any cost or context-length estimate assuming a clean word-to-token ratio should be treated with real caution

Likely follow-ups: Why does the same context window "fit less" for code than for plain English text? How does tokenization affect a model's ability to do character-level tasks, like counting letters in a word? **Red flag:** treating "tokens" and "words" as interchangeable when estimating cost or context usage. **Learn it in:** Chapter 54 (Generative AI & Large Language Models).

Q79-002 · What's an embedding, and why does cosine similarity, not Euclidean distance, dominate as the comparison metric for them?

Remember it as: Cosine similarity asks "do these two vectors point in the same direction," ignoring how long each vector is. For meaning-based comparison, direction is usually what actually matters, not magnitude.

Answer in one line: An **embedding** is a dense numeric vector representing a piece of text (or an image, audio, etc.) such that semantically similar inputs produce vectors that are close together in that vector space; **cosine similarity** measures the angle between two vectors, ignoring their magnitude, which tends to matter more for text length or intensity than for semantic meaning, making angle a more reliable meaning-similarity signal than raw distance.

Verified, live:

```
def cosine_sim(a, b):
    return np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b))

query   = [0.9, 0.1, 0.0] # a query about "returns"
doc_a   = [0.85, 0.15, 0.05] # a document actually about returns
doc_b   = [0.1, 0.2, 0.95] # a document about something unrelated
```

```
similarity to relevant doc: 0.996
similarity to irrelevant doc: 0.124
```

Tier	What to say
Passes	"Embeddings capture meaning as numbers" (correct, no explanation of the actual comparison mechanism)
Strong	The direction-vs-magnitude distinction above, correctly identifying why cosine similarity is the standard choice for embedding comparison
Extra points	+ [Validate] the real computed similarity scores above: 0.996 for a genuinely related pair, 0.124 for an unrelated one, a clean, large separation that's exactly what makes retrieval work reliably + [Depth] this same dot-product-based similarity is the identical mathematical operation Chapter 35 introduces for the dot product itself, embeddings are one of its highest-leverage real-world applications

Likely follow-ups: What happens to retrieval quality if two genuinely different documents happen to produce very similar embeddings? How would you visualize a set of embeddings to sanity-check they're capturing meaningful structure? **Red flag:** not knowing why cosine similarity, specifically, is the standard choice over a plain distance metric. **Learn it in:** Chapter 54 and Chapter 35, §35.1 (the dot product).

Rapid-fire, 79.1

#	Question	One-line answer	Extra point
Q79-003	What's a context window, in plain terms?	The maximum amount of text (measured in tokens) a model can consider at once, including both the prompt and its own generated response	[Business] a conversation or document exceeding the context window must be truncated or summarized somehow, or the model simply can't see the parts that don't fit
Q79-004	What's temperature, in an LLM's generation settings?	A setting controlling how random or deterministic the model's next-token choice is; low temperature favors the most likely next token consistently, high temperature allows more varied, creative, less predictable output	[Business] a factual extraction task usually wants low temperature (consistency); a creative writing task often wants it higher
Q79-005	What's the difference between fine-tuning and prompting (few-shot examples) as ways to adapt a model's behavior?	Fine-tuning updates the model's own weights on new examples, a heavier, more permanent change; prompting adapts behavior at inference time through instructions and examples in the prompt itself, no weight changes at all	[Trade-offs] fine-tuning can achieve a more consistent, reliable behavior change for a narrow task, at real cost and complexity; prompting is far cheaper to iterate on and is often sufficient
Q79-006	What's a multimodal model?	A model that can process and/or generate more than one type of input or output, text and images together, for instance, rather than being restricted to text alone	[Learn it in] Chapter 54

79.2 Basic-but-tricky GenAI questions

Q79-007 · If an LLM "hallucinates" a confident, plausible-sounding wrong answer, is that a bug in the model?

Remember it as: A language model is fundamentally predicting a plausible next token, not looking up a verified fact. Hallucination isn't a malfunction, it's the expected behavior of that mechanism when it has no grounding to lean on.

Answer in one line: Not exactly a "bug" in the traditional sense: an LLM generates text by predicting statistically plausible continuations, and when it lacks reliable information about a specific fact, it can still generate fluent, confident-sounding text that's simply wrong, since fluency and factual accuracy

are not the same thing the model is optimizing for; this is precisely why grounding techniques like RAG (§79.3) exist, to give the model real, retrieved information to base its answer on instead of relying purely on its trained-in, sometimes-wrong "memory."

Tier	What to say
Passes	"Hallucination means the model makes things up sometimes" (true, no explanation of the underlying mechanism)
Strong	The plausibility-vs-accuracy distinction above, connecting it directly to why RAG and grounding are the standard mitigation
Extra points	+ [Business] a system with no grounding and no way to say "I don't know" will always eventually hallucinate on a question outside its reliable knowledge; the fix isn't a "smarter" model alone, it's an architecture (retrieval, citations, confidence signaling) that gives the model a real alternative to guessing

Likely follow-ups: How would you measure a system's hallucination rate in practice? What's the risk of a RAG system hallucinating even *with* retrieved context available? **Red flag:** treating hallucination as a simple defect that better prompting alone fully solves. **Learn it in:** Chapter 55 (RAG, agents, and evaluation).

Rapid-fire, 79.2

#	Question	One-line answer	Extra point
Q79-008	Does a bigger context window mean you should always stuff in as much context as possible?	No: more context can dilute the model's attention across irrelevant information, sometimes degrading answer quality even when the truly relevant information is technically present somewhere in the prompt	[Business] relevant, well-curated context (via good retrieval, §79.3) usually beats simply maximizing context volume
Q79-009	Is prompt engineering a durable, transferable skill, or is it fragile to the specific model?	Some general principles transfer (clarity, structure, examples), but specific prompt wording that works well on one model version can behave differently on another, an ongoing maintenance cost worth planning for	[Business] a production system built on carefully tuned prompts needs regression testing (Chapter 57) when the underlying model is updated, exactly like code needs tests when a dependency updates
Q79-010	Can you fully prevent prompt injection in a system that takes user input and feeds it to an LLM?	Not with complete certainty using prompting alone; layered defenses (input validation, output filtering, limiting what the model is allowed to actually do) reduce risk but don't eliminate it entirely	[Learn it in] Chapter 57 (LLMOps: security, prompt injection)
Q79-011	Is an LLM-based system deterministic, given the exact same input twice?	Not necessarily, even at temperature 0, due to factors like floating-point non-determinism in the underlying computation across different hardware or batching; treat outputs as close-to-deterministic, not guaranteed byte-identical	[Edge cases] this matters for testing: an exact-match regression test on LLM output can be more brittle than a semantic-similarity-based check

79.3 Retrieval-Augmented Generation (RAG)

Q79-012 · Walk through RAG end to end, and implement a simple chunking function

Remember it as: Retrieve first, then generate. The model only answers using what was actually found, not purely from what it happened to memorize during training.

Answer in one line: RAG breaks source documents into **chunks**, converts each chunk into an **embedding**, stores them in a **vector index**; at query time, the query is embedded and compared against that index to **retrieve** the most relevant chunks, which are then inserted into the model's prompt as grounding context before it **generates** an answer, ideally with **citations** back to the source chunks used.

Verified, live, a real fixed-size chunking function with overlap:

```
def chunk_text(text, chunk_size=60, overlap=15):
    chunks = []
    start = 0
    while start < len(text):
        end = start + chunk_size
        chunks.append(text[start:end])
        start += chunk_size - overlap
    return chunks
```

```
chunk 0: "Riverstone's return policy allows customers to return unused"
chunk 1: "o return unused items within 30 days of purchase for a full "
chunk 2: "ase for a full refund, provided the original packaging is in"
chunk 3: "packaging is intact."
```

Tier	What to say
Passes	Names the RAG pipeline stages in the right order, can't produce working chunking logic
Strong	The working overlap-chunking function above, correctly explaining <i>why</i> overlap exists: without it, a sentence split exactly at a chunk boundary can lose meaning that only makes sense with the text on the other side of the cut
Extra points	+ [Validate] the real output above shows the overlap in action: "return unused" appears at the end of chunk 0 and the start of chunk 1, exactly the redundancy meant to preserve meaning across the cut + [Edge cases] fixed-size chunking (shown here) is simple but can still split a sentence or table row awkwardly; semantic chunking (splitting on natural boundaries like paragraphs or sections) is a common refinement when document structure allows it

Likely follow-ups: How would you choose chunk size for a specific document type (a legal contract vs. a product FAQ)? What's reranking, and why is it often added after the initial retrieval step? **Red flag:** describing RAG's stages with no understanding of why chunking or overlap specifically matters. **Learn it in:** Chapter 55 (RAG end to end).

Q79-013 · What's reranking, and why does an extra step after initial vector retrieval improve answer quality?

Remember it as: The first retrieval pass is fast but approximate. Reranking is a slower, more careful second look at just the top candidates, worth the extra cost because there are far fewer of them to examine closely.

Answer in one line: Initial vector retrieval (cosine similarity search over an embedding index) is fast but approximate, optimized to quickly narrow millions of chunks down to a top handful; a **reranker**, typically a more computationally expensive model built to score relevance directly, then re-scores just that smaller candidate set more carefully, often surfacing a better final ranking than the fast initial pass alone.

Tier	What to say
Passes	"Reranking makes the results better" (true, no explanation of why a two-stage approach specifically)
Strong	The speed-vs-accuracy two-stage trade-off above: cheap-and-broad first pass, expensive-and-precise second pass on a much smaller set
Extra points	+ [Business] this two-stage pattern (fast filter, then expensive precise scoring on a small candidate set) is a general systems-design idea that recurs well beyond RAG, wherever an expensive precise method doesn't scale to the full dataset but is affordable on a pre-filtered handful of candidates

Likely follow-ups: What would you check if reranking made results measurably worse on a specific query? How would you decide how many candidates to pass into the reranking stage? **Red flag:** treating reranking as an optional polish step rather than understanding the actual two-stage cost/accuracy trade-off it exists to solve. **Learn it in:** Chapter 55.

Rapid-fire, 79.3

#	Question	One-line answer	Extra point
Q79-014	Why does a RAG system need citations, not just an answer?	Citations let a user (or an automated evaluator) verify the answer actually traces back to real, retrieved source material, rather than trusting the model's fluency alone	[Learn it in] Chapter 55's grounding and citation discipline
Q79-015	What happens if the retrieval step finds no genuinely relevant chunks for a query?	A well-designed system should recognize this and respond with an honest "I don't have information on that" rather than generating a plausible-sounding answer from the model's untethered training knowledge	[Learn it in] Q79-007 (hallucination without grounding)
Q79-016	Why might a RAG system need to periodically re-embed and re-index its documents?	If the underlying embedding model is updated, or the source documents change meaningfully, the existing vector index can become stale or inconsistent with newly embedded content	[Business] this is a real, recurring maintenance cost of a RAG system, not a one-time setup task
Q79-017	What's a vector database, and how is it different from a traditional relational database for this use case?	Optimized specifically for fast approximate nearest-neighbor search over high-dimensional vectors, a query pattern a traditional relational database's indexing isn't built for at scale	[Learn it in] Chapter 71's indexing discussion (§71.10) covers the relational-database side of this same underlying "how do you search fast at scale" problem

79.4 Agents and tool use

Q79-018 · What's the difference between a simple prompt-and-response LLM call and an "agent"?

Remember it as: A single call answers once. An agent can decide to call a tool, look at the result, and decide again, in a loop, until it's actually done.

Answer in one line: A single LLM call takes a prompt and returns one response; an **agent** wraps an LLM in a loop that lets it decide to call external tools (a search, a calculator, a database query, an API), observe the result, and decide on a next action, potentially several times, before producing a final answer, giving it the ability to take multi-step actions rather than just generate text once.

Tier	What to say
Passes	"An agent is an LLM that can use tools" (correct, no mention of the decision loop that makes multi-step behavior possible)
Strong	The observe-decide-act loop above, correctly distinguishing single-shot generation from iterative, tool-using behavior
Extra points	+ [Business] an agent's added capability comes with added failure modes: it can call the wrong tool, misinterpret a tool's result, or loop unproductively, all failure modes a single prompt-response call simply doesn't have, which is exactly why evaluating and guardrailing agents (§79.5) is a genuinely harder problem than evaluating a single response

Likely follow-ups: What's the risk of an agent getting stuck in a loop, and how would you guard against it? How would you decide whether a task actually needs an agent versus a single well-designed prompt? **Red flag:** treating "agent" as just a marketing term with no specific technical distinction from a single LLM call. **Learn it in:** Chapter 55 (tool use and agents).

Rapid-fire, 79.4

#	Question	One-line answer	Extra point
Q79-019	Why does an agent typically need a maximum number of steps or a timeout?	Without one, a genuinely stuck agent (repeatedly calling the same tool, or oscillating between two actions) can loop indefinitely, wasting cost and never returning a result	[Learn it in] Chapter 78, Q78-021 (a circuit breaker is the identical safety mechanism, applied here to agent loops instead of API retries)
Q79-020	What's the risk of giving an agent access to a tool with real side effects (sending an email, modifying a database)?	An agent's decision to call that tool is itself a probabilistic model output, not a guaranteed-correct decision; a wrong or hallucinated tool call with real side effects can cause real, hard-to-undo damage	[Business] high-stakes, irreversible actions often warrant a human-in-the-loop confirmation step before an agent executes them
Q79-021	How would you debug an agent that's producing a wrong final answer despite calling the right tools?	Trace through the full sequence: what each tool actually returned, and how the agent interpreted that result at each step, since the failure could be in tool selection, tool execution, or the agent's interpretation of a correct tool result	[Learn it in] Chapter 77's data lineage discipline (Q77-030), the same "trace backward through the steps" debugging method

#	Question	One-line answer	Extra point
Q79-022	What's the difference between a single agent and a multi-agent system?	A single agent handles the whole task itself, looping through tool calls; a multi-agent system splits the task across multiple specialized agents that communicate, often trading simplicity for the ability to specialize each agent on a narrower, better-scoped role	[Trade-offs] more agents means more coordination complexity and more places for a miscommunication between agents to cause a wrong final outcome

79.5 Evaluating AI applications

Q79-023 · How do you evaluate an LLM-based application's output quality, given there's often no single "correct" answer to check against?

Remember it as: A golden set gives you known-good answers to check against. An LLM-as-judge extends that same idea to open-ended answers a simple string match can't score.

Answer in one line: Build a **golden set** of representative queries with known-good expected answers (or key facts that must appear) to check against systematically; for genuinely open-ended output where an exact match doesn't make sense, use **LLM-as-judge** (a separate model call scoring the output against defined criteria) alongside periodic **human review**, since no single method alone reliably catches every failure mode.

Tier	What to say
Passes	"You'd just check if the answers look good" (no systematic method at all)
Strong	The golden-set-plus-LLM-as-judge-plus-human-review combination above, correctly noting no single method is sufficient alone
Extra points	+ [Edge cases] an LLM-as-judge has its own failure modes (it can be fooled by fluent-but-wrong answers, or have its own biases about what "good" looks like), so it needs periodic calibration against real human judgment, not treated as a fully independent ground truth + [Business] a golden set needs to be actively maintained and expanded as new failure modes are discovered in production, not built once and left static

Likely follow-ups: How would you build an initial golden set with no existing production data to draw from? What's the risk of over-optimizing a system specifically to score well on its own golden set?

Red flag: relying entirely on spot-checking outputs by eye, with no systematic, repeatable evaluation method. **Learn it in:** Chapter 55 (evaluating AI applications).

Rapid-fire, 79.5

#	Question	One-line answer	Extra point
Q79-024	What's a guardrail, in the context of an LLM application?	A check (before or after the model's generation) that catches and blocks unsafe, off-topic, or otherwise unacceptable output before it reaches the end user	[Business] guardrails are a defense layer separate from prompting; relying on prompting alone to prevent bad output is fragile, the same lesson as Q79-010's prompt-injection limits

#	Question	One-line answer	Extra point
Q79-025	Why might an evaluation metric that looks good in aggregate still hide a real problem?	The same Simpson's-paradox-style risk (Chapter 73, Q73-031): an overall-good average score can mask a specific query type or user segment the system handles consistently poorly	[Learn it in] Chapter 73, Q73-031
Q79-026	What's a regression test for an LLM-based system, specifically?	A saved set of prompts and their previously-acceptable outputs, re-run whenever the underlying model, prompt, or retrieval logic changes, to catch a quality regression before it reaches production	[Learn it in] Chapter 57 (prompt versioning and regression tests)
Q79-027	Why is human review still necessary even with a good automated evaluation pipeline in place?	Automated methods (golden sets, LLM-as-judge) can miss failure modes nobody anticipated when designing them; periodic human review is what catches the genuinely novel problem the automated system wasn't built to look for	[Business] this mirrors Chapter 77's observability-vs-quality-checks distinction: automated checks catch what you anticipated, human review is closer to genuine observability

79.6 MLOps: making models survive production

Q79-028 · Detect whether a production model's input data has genuinely drifted from its training data, and prove it statistically

Remember it as: "It looks a little different" isn't drift detection. A statistical test that compares two distributions and gives you a real *p*-value is.

Answer in one line: Use a statistical test comparing the training-time distribution of a feature against its current production distribution, a **Kolmogorov-Smirnov (KS) test** for a continuous feature, or a population stability index for a binned/categorical one, rather than eyeballing summary statistics or a chart.

Verified, live:

```
from scipy import stats

training_dist = rng.normal(50, 10, 1000)
prod_same     = rng.normal(50, 10, 1000)    # genuinely no real shift
prod_drifted  = rng.normal(58, 10, 1000)    # a real mean shift, 50 -> 58

stats.ks_2samp(training_dist, prod_same)    # -> KS stat 0.029, p = 0.795
stats.ks_2samp(training_dist, prod_drifted) # -> KS stat 0.317, p = 0.000000
```

The test correctly distinguishes the two cases: no real drift gives a high *p*-value (0.795, not statistically distinguishable), while a genuine shift gives an extremely low *p*-value (essentially 0), a clean, real demonstration of the test actually working as intended.

Tier	What to say
Passes	"You'd monitor the model's accuracy over time" (a lagging signal; drift in <i>input features</i> can often be detected before it shows up as a performance drop at all)

Tier	What to say
Strong	The KS-test approach above, monitoring input feature distributions directly, which can catch drift before it ever degrades a downstream metric
Extra points	+ [Validate] the real, clean separation above between the no-drift and real-drift cases proves the method actually works, not just that it exists + [Business] input-drift detection is a <i>leading</i> indicator (Chapter 75, Q75-014) for a model's health, while accuracy degradation is a <i>lagging</i> one; catching drift early, before it shows up in outcomes, is exactly the value this kind of monitoring adds

Likely follow-ups: What would you do once drift is detected, retrain immediately, or investigate first? How would you monitor drift for a categorical feature instead of a continuous one? **Red flag:** relying solely on downstream accuracy metrics to detect drift, missing the chance to catch it earlier via the input features themselves. **Learn it in:** Chapter 56 (MLOps: drift detection).

Rapid-fire, 79.6

#	Question	One-line answer	Extra point
Q79-029	What's a model registry, and why does a team need one?	A centralized system tracking every trained model version, its metrics, and its deployment status, so "which exact model is live in production right now" always has one clear, authoritative answer	[Business] without one, a team can lose track of exactly which model version produced a specific production prediction, a real problem when debugging a bad outcome later
Q79-030	What's a feature store, and what problem does it solve?	A centralized system for computing and serving features consistently between training and production serving, directly addressing training-serving skew (Chapter 74, Q74-030)	[Learn it in] Chapter 74, Q74-030
Q79-031	Why does a model need a defined retraining trigger, rather than retraining on a fixed calendar schedule alone?	A fixed schedule might retrain too often (wasting compute on a model that hasn't actually degraded) or too rarely (leaving a genuinely drifted model live for too long); a trigger tied to actual detected drift or performance decline responds to the real signal instead of an arbitrary calendar date	[Business] the best setups often combine both: a scheduled retrain as a baseline cadence, plus an event-triggered retrain when drift is detected outside that schedule
Q79-032	What's the difference between online and batch model serving?	Online serving responds to individual requests in real time (a single prediction per API call); batch serving scores a large dataset all at once on a schedule, with no per-request latency requirement at all	[Business] the choice depends on whether a downstream decision genuinely needs a fresh, on-demand prediction (online) or can work from a periodically refreshed batch of scores (batch), the same freshness-driven logic as Q77-002's batch-vs-streaming pipeline decision

79.7 LLMOps

Q79-033 · Estimate the daily cost of an LLM-powered feature, and explain what levers you'd pull if it's too expensive

Remember it as: Cost scales with tokens in, tokens out, and calls per day, so any lever that reduces one of those three reduces cost proportionally.

Answer in one line: Cost per call is driven by input tokens, output tokens, and the per-token price for each (input and output are often priced differently), multiplied by the number of calls per day; to reduce cost, shrink the prompt (better retrieval, less unnecessary context), shrink the expected output length, cache repeated or similar queries, or route simpler queries to a cheaper, smaller model.

Verified, live:

```
def estimate_cost(input_tokens, output_tokens, input_price_per_1k, output_price_per_1k):  
    return (input_tokens/1000)*input_price_per_1k + (output_tokens/1000)*output_price_per_1k  
  
cost_per_call = estimate_cost(500, 150, 0.15, 0.60)    # illustrative rates
```

```
cost per call: $0.1650  
daily cost at 2,000 calls/day: $330.00
```

Tier	What to say
Passes	"LLM calls cost money based on usage" with no structured way to estimate or reduce it
Strong	The token-based cost formula above, correctly separating input and output token pricing, and naming at least two concrete cost-reduction levers
Extra points	+ [Validate] the real calculation above turns an abstract "it costs money" into a concrete daily number a business can actually evaluate against the feature's value + [Business] caching deserves special mention: for a feature with many repeated or near-identical queries, a cache hit costs nothing, and can be the single highest-leverage cost lever, higher than model choice alone, for a query pattern with real repetition

Likely follow-ups: How would you decide which queries are safe to route to a cheaper model versus which need the more capable one? What's the trade-off of caching for a use case where answers need to reflect very current information? **Red flag:** no structured cost model at all, or ignoring caching as a lever entirely. **Learn it in:** Chapter 57 (LLMOps: cost per request, routing, and caching).

Rapid-fire, 79.7

#	Question	One-line answer	Extra point
Q79-034	Why does an LLM application need prompt versioning, the same way code needs version control?	A prompt change can meaningfully change output quality or behavior; without versioning, there's no way to know exactly which prompt produced a given historical output, or to safely roll back a change that made things worse	[Learn it in] Q79-026

#	Question	One-line answer	Extra point
Q79-035	What's tracing, in an LLM application context?	Recording the full sequence of steps (retrieval, tool calls, model calls, intermediate outputs) behind a single final response, so a specific bad output can be debugged step by step instead of treated as an unexplainable black box	[Learn it in] Q79-021 (the same trace-the-steps debugging discipline for agents)
Q79-036	What's the risk of routing a query to a cheaper, smaller model purely based on cost, with no quality check?	The cheaper model may produce a meaningfully worse answer for that specific query's complexity, silently degrading the user's experience in exchange for a cost saving that may not be worth it	[Business] any routing strategy needs its own evaluation (§79.5) proving the quality trade-off is actually acceptable, not just an assumed one
Q79-037	Why is prompt injection a genuinely different security problem from a traditional SQL injection?	Traditional injection exploits a rigid parser with a fixable grammar; prompt injection exploits a model's flexible, natural-language interpretation, which has no equally rigid, fully closeable boundary between "instructions" and "data" the way a parameterized SQL query does	[Learn it in] Q79-010

79.8 Full design cases

Q79-038 · Design a RAG-based product-support assistant for Riverstone, end to end

What they're really testing: whether a complete RAG system design comes out under time pressure, covering ingestion, retrieval, generation, and evaluation, not just the retrieval step alone.

Talked through live, start to finish:

"Source content: product manuals, past support tickets, and the returns policy, ingested and chunked (Q79-012), each chunk embedded and stored in a vector index. At query time: a customer's question is embedded, the index returns the top-k most similar chunks, and a reranker (Q79-013) narrows those to the genuinely most relevant few before they're inserted into the prompt as grounding context. The model generates an answer citing which source chunks it used, and if retrieval finds nothing sufficiently relevant, the system explicitly says it doesn't have a confident answer rather than guessing (Q79-015), routing to a human agent instead. For evaluation, I'd build a golden set from real historical support tickets with known-good resolutions, checked with LLM-as-judge plus periodic human review (Q79-023), and I'd add a guardrail blocking any generated answer that promises something outside Riverstone's actual policies, like an unauthorized discount. For cost, I'd cache answers to genuinely repeated common questions (Q79-033), since support questions have real repetition across customers."

Extra-points moves demonstrated: **[Structure]** covered ingestion through evaluation and cost, not just the retrieval mechanics. **[Edge cases]** explicitly handled the no-good-retrieval case with a human handoff rather than letting the model guess. **[Business]** named a guardrail tied to a real business risk (an unauthorized discount promise), not a generic safety placeholder.

Likely follow-ups: How would you handle a question that spans multiple product manuals? What would make you decide this needs an agent (able to look something up dynamically) rather than pure RAG? **Red flag:** a design that only covers retrieval and generation, with no mention of evaluation, guardrails, or the no-good-answer case. **Learn it in:** §79.3 and §79.5 above, pulled together.

Q79-039 · Design a model-serving platform that supports both real-time and batch predictions for multiple models

What they're really testing: whether the online-vs-batch distinction (Q79-032) gets applied to a genuine shared-infrastructure design, not treated as two unrelated systems.

Talked through live, start to finish:

"A model registry (Q79-029) is the shared source of truth for every model version and its deployment status, used by both serving paths. For real-time serving: models loaded behind a low-latency API, with the feature store (Q79-030) ensuring the exact same feature computation logic used in training is used at serving time, avoiding training-serving skew. For batch serving: a scheduled job scores a full dataset on a cadence appropriate to that model's use case (Chapter 77's batch pipeline pattern applies directly here). Both paths log predictions with the exact model version used, feeding both the registry's own tracking and the drift-detection monitoring (Q79-028) that watches each live model's input distribution over time. A retraining trigger (Q79-031) fires either on a baseline schedule or when drift crosses a defined threshold, whichever comes first."

Extra-points moves demonstrated: [Depth] unified both serving paths under one shared registry and feature store rather than designing them as two disconnected systems. [Business] connected monitoring directly to a concrete retraining trigger, closing the loop rather than leaving monitoring as a dashboard nobody acts on.

Likely follow-ups: How would you handle two models needing genuinely different feature computation logic within the same shared feature store? What would you monitor differently for the batch path versus the real-time path? **Red flag:** designing the two serving paths as entirely separate systems with no shared registry, feature logic, or monitoring. **Learn it in:** Chapter 56 (MLOps) and Chapter 77 (the batch pipeline pattern this design's batch path reuses directly).

Common mistakes and how to spot them

Mistake	Symptom	Fix
Assuming a bigger context window means "always add more context"	Diluted attention, sometimes worse answers despite more information present	Curate relevant context via good retrieval, don't maximize volume
No grounding, relying purely on the model's trained-in knowledge	Confident, fluent, sometimes wrong answers (hallucination)	Use RAG to ground answers in real, retrieved, current information

Mistake	Symptom	Fix
Chunking with no overlap	Meaning split across a chunk boundary gets lost	Use overlapping chunks, or semantic chunking on natural boundaries
Monitoring only downstream accuracy, not input drift	Drift goes undetected until it's already hurt real outcomes	Statistically test input feature distributions directly (Q79-028)
No cost model for an LLM feature before shipping it	An unpleasant cost surprise once usage scales up	Estimate cost per call and multiply by expected volume before launch
An agent given side-effect tools with no human-in-the-loop check	A wrong or hallucinated tool call causes real, hard-to-undo damage	Require confirmation for high-stakes, irreversible actions
Evaluating only on an aggregate score	A real, segment-specific failure mode hides behind a good average	Check performance broken out by query type or user segment

In the real world: the RAG assistant that confidently cited a document that didn't say what it claimed

Priyanka, a Data Scientist, is asked in an interview to describe debugging a real RAG system issue. She describes a customer-support assistant that had started giving customers an incorrect return window, and the response even included a citation to a real, correctly retrieved policy document, which made the error harder to spot at first: the citation looked legitimate, so nobody initially suspected the retrieved content itself.

She traces it to the chunking boundary: the policy document's actual text read "returns are accepted within 30 days, except for electronics, which have a 14-day window," and a fixed-size chunk had split exactly between those two clauses, so the retrieved chunk for an electronics question contained only "returns are accepted within 30 days," the exception clause landing entirely in the next chunk, which wasn't retrieved for that specific query. The citation was technically accurate, that chunk really did come from the real policy document, but the chunk itself no longer contained the complete, correct information.

The interviewer's follow-up: how did she fix it, and how did she make sure it wouldn't happen again the same way? Her answer: she moved to semantic chunking on paragraph boundaries instead of a fixed character count, so a chunk wouldn't split a sentence's meaning apart from its own exception clause, and she added a golden-set test case specifically covering the electronics exception, so a future chunking or retrieval change that reintroduced this exact failure would be caught automatically before shipping, not discovered again by a customer complaint.

The interviewer's note: *"A citation being real doesn't mean the retrieved content is complete. She caught a failure mode a simpler 'is this grounded' check would have completely missed."* That's the deeper lesson this whole chapter is built around: grounding and citations reduce hallucination risk, but they don't eliminate every way a RAG system can still confidently produce a wrong answer.

Tools

Python, **NumPy**, and **SciPy**, used throughout this chapter for every computable claim (cosine similarity, chunking, the KS drift test). In real GenAI/MLOps work: a vector database (for RAG indexing), an orchestration framework for agents, MLflow or a similar tool for model registry and tracking, and an LLM observability/tracing platform. Vendor and model specifics change quickly; verify current details before relying on them in an actual interview.

The project

Goal: apply this chapter's verifiable pieces to a real or realistic scenario.

1. Implement the chunking function from Q79-012 on a real document of your own, and inspect a chunk boundary to check whether it splits meaning awkwardly, the way this chapter's real-world story describes.
 2. Compute cosine similarity between a real query and a few candidate documents (using any embedding source available to you) and confirm the ranking matches your own judgment of relevance.
 3. Run the KS-test drift-detection pattern from Q79-028 on two real or simulated datasets of your own, one with a genuine shift and one without, and confirm the test correctly distinguishes them.
 4. Estimate the daily cost of a hypothetical LLM feature for your own context, using Q79-033's formula, and name the one cost lever you'd pull first if it came in too expensive.
-

Final-week revision list

Q79-001, Q79-002, Q79-007, Q79-012, Q79-013, Q79-018, Q79-023, Q79-028, Q79-033, Q79-038, Q79-039.

Key terms

token · context window · temperature · fine-tuning vs. prompting · embedding · cosine similarity · hallucination · RAG (Retrieval-Augmented Generation) · chunking (fixed-size vs. semantic) · vector database · reranking · citation/grounding · agent · tool use · golden set · LLM-as-judge · guardrail · regression test (LLM) · model registry · feature store · drift detection · KS test · retraining trigger · online vs. batch serving · prompt versioning · tracing (LLM) · prompt injection · caching (LLM cost)

Where this leads

- **Chapter 69, The Extra-Points Method**, is the rubric and move set every answer above is written against.
- **Chapter 74, Machine Learning Question Bank**, already covers training-serving skew and drift-adjacent concepts (Q74-030) from the classical-ML angle; this chapter extends the identical discipline to LLM-based systems.
- **Chapter 77, Data Engineering & Data System Design Bank**, supplies the batch pipeline and circuit-breaker patterns this chapter's design cases reuse directly for the batch-serving path and agent-loop safety.
- **Chapters 54–57 of this book** teach every technique this bank draws on, in full; this chapter tests it, it doesn't re-teach it from scratch.